Lab 03 — Kubernetes Internal Workflow — From kubectl to runc

ID: K8S-LAB-03 · **Category:** Runtime · **Level:** Engineer · **Duration:** 60 min

Focus: api-server, scheduler, kubelet, containerd, runc, reconciliation

Objective

Trace a single `kubectl apply` through every control-plane and node component until a Linux process is running.

Concepts Covered

• Desired state vs actual state (the reconciliation loop)

• How the scheduler ranks nodes

• How kubelet uses the CRI to talk to containerd

• Where namespaces and cgroups actually get created

Prerequisites

• A cluster from Lab 1

Architecture

Every component just watches the API server and reacts. There's no central coordinator — only loops converging the actual state onto the desired state stored in etcd.

Lab Steps

01. Apply a Deployment and watch events in another terminal

You'll see Scheduled → Pulling → Pulled → Created → Started in order.

```
kubectl create deployment trace --image=nginx --replicas=1
# in another terminal:
kubectl get events --watch
```

Expected: Five events within ~5s on a warm node.

Internals: Each event is emitted by a different component: scheduler, kubelet, then runtime.

02. Inspect the scheduling decision

Shows which node was picked and which scheduler made the call.

```
kubectl describe pod -l app=trace | grep -A2 Scheduled
```

Expected: `Successfully assigned default/trace-... to <node>` by default-scheduler.

Internals: Scheduler runs predicates (filter) then priorities (score). Final winner is bound via a Pod Bind subresource update.

03. See the container on the node

Proves the abstraction: there's just a Linux process under all the YAML.

```
sudo crictl ps | grep nginx
```

Expected: A container with state Running.

Internals: crictl talks the same CRI protocol kubelet uses.

04. Inspect the cgroup

The Pod gets a cgroup slice; the container is a unit inside it.

```
sudo systemd-cgls | head -30
```

Expected: A `kubepods.slice/...` hierarchy.

Internals: cgroups enforce CPU/memory limits; without them `limits:` in the Pod spec would be advisory.

05. Watch self-healing in real time

Deleting the Pod doesn't kill the workload — the ReplicaSet observes a missing Pod and asks the API for a new one.

```
kubectl delete pod -l app=trace
kubectl get pods -l app=trace -w
```

Expected: New Pod created within ~2s.

Internals: ReplicaSet controller watches Pods matching its selector and posts a new Pod when count < replicas.

Common Errors

Pod stays Pending forever — Cause: No node matches the requests/tolerations · Fix: `kubectl describe` → Events reveal which predicate failed

Pod created twice after `kubectl delete` — Cause: User deleted the Pod but a Deployment owns the ReplicaSet · Fix: Expected behavior — delete the Deployment, not the Pod

Troubleshooting

• If the scheduler is silent, check it's running: `kubectl -n kube-system get pods | grep scheduler`

• If pods never start, watch the runtime: `sudo crictl events`

Enterprise Relevance

Understanding the loop is what makes you trust Kubernetes in production: there is no magic, only convergent controllers.

Validation

• You can name every component that touches a Pod from kubectl to runc

• You can predict whether a manual `kubectl delete pod` will respawn or not

Related Incidents & Guide Sections

Incidents: K8S-INC-3004

• Control plane & worker components

• Pod lifecycle & probes

Questions

Beginner

Q. What does the scheduler actually do?

A. Picks a node for an unbound Pod and writes a Binding back to the API.

Intermediate

Q. What ties a container back to its Pod on the node?

A. The sandbox (pause) container shares its network/IPC namespaces with all app containers in the Pod.

Advanced

Q. Why is reconciliation eventually consistent?

A. Controllers act on watched events with retries; partial failures heal on the next sync rather than being transactional.

Scenario

Q. etcd disk fills up. What's the first symptom you'll see?

A. API server starts returning 500s on writes; reads may still work briefly.

Viva

Q. Where do static pods come from?

A. Files in /etc/kubernetes/manifests/, watched directly by kubelet, not scheduled.